

Kristine Stanley

Senior Software Engineer | Python & React & Cloud

Coaldale, CO 81222 | kristine.stanley.dev@outlook.com | +1 (786) 949-7561 | <https://www.linkedin.com/in/kristine-stanley-341688411>

PROFESSIONAL SUMMARY

Senior Software Engineer with 11+ years delivering cybersecurity, connected-mobility, insurance, and learning-platform products using **Python 3.4–3.12**, **FastAPI**, **Django**, **React 14–19**, **TypeScript**, **PostgreSQL**, **AWS**, and **Azure**; combines backend architecture, high-performance frontend engineering, secure API design, event-driven processing, cloud modernization, observability, and technical leadership to improve reliability, release quality, developer productivity, and customer outcomes.

SKILLS

- **Languages:** Python 3.4–3.12, TypeScript 3–5, JavaScript ES6+, SQL, Bash, HTML5, CSS3
- **Frontend:** React 14–19, Next.js 12–15, Redux Toolkit, TanStack Query, React Router, React Hook Form, Zustand, Context API, Server Components, Tailwind CSS, Material UI, Storybook
- **Python Backend:** FastAPI, Django, Django REST Framework, Flask, Pydantic, SQLAlchemy, Alembic, Celery, asyncio, aiohttp, Pandas
- **API Technologies:** REST, GraphQL, OpenAPI, Swagger, WebSockets, Server-Sent Events, webhooks, OAuth 2.0, OpenID Connect, JWT, API versioning
- **Architecture:** Microservices, modular monoliths, event-driven architecture, domain-driven design, hexagonal architecture, CQRS, transactional outbox, backend-for-frontend, multi-tenant systems
- **Databases:** PostgreSQL, MySQL, SQL Server, MongoDB, DynamoDB, Redis, schema design, indexing, query optimization, migrations, connection pooling
- **Messaging:** Apache Kafka, RabbitMQ, AWS SQS, Azure Service Bus, Celery workers, dead-letter queues, retry policies, event replay, idempotent processing
- **Cloud:** AWS Lambda, ECS, EKS, API Gateway, S3, RDS, DynamoDB, SQS, CloudWatch; Azure Functions, App Service, AKS, Service Bus, Key Vault
- **Testing:** Pytest, unittest, Hypothesis, Jest, React Testing Library, Playwright, Cypress, MSW, Pact, Testcontainers, Locust
- **DevOps:** Docker, Kubernetes, Terraform, Helm, GitHub Actions, Azure DevOps, Jenkins, Vite, Webpack, Nx, CI/CD pipelines
- **Monitoring:** OpenTelemetry, Datadog, Prometheus, Grafana, ELK Stack, Sentry, CloudWatch, structured logging, distributed tracing
- **Leadership:** Architecture reviews, technical mentoring, code reviews, production support, incident response, root-cause analysis, Agile delivery, stakeholder communication

EXPERIENCE

Veracode Systems Inc — Senior Software Engineer

Remote/ Full time

April 2021 – May 2026

Somerville, MA

- Led architecture and delivery for enterprise cybersecurity products using **Python 3.8–3.12**, **FastAPI**, **Django**, **React 16–19**, **TypeScript**, and **PostgreSQL** across vulnerability management, policy evaluation, analytics, reporting, and remediation workflows.
- Replaced tightly coupled Django modules and duplicated React state logic with domain-oriented services, typed **Pydantic** contracts, reusable hooks, and **TanStack Query**, reducing recurring production defects by **36%** across multiple product teams.
- Resolved dashboard freezes and slow security APIs by combining virtualized tables, cursor pagination, composite database indexes, Redis caching, and optimized response models, improving P95 end-to-end interaction time by **44%**.

- Modernized legacy Python services and class-based React applications through incremental runtime upgrades, strict TypeScript and Python typing, functional components, asynchronous clients, and automated compatibility testing.
- Designed secure multi-tenant APIs with **FastAPI**, OAuth 2.0, OpenID Connect, JWT validation, tenant-scoped repositories, and policy-based authorization that prevented unauthorized access to customer vulnerability and compliance data.
- Eliminated duplicated security findings caused by message retries through idempotency keys, transactional outbox processing, **Kafka** consumer checkpoints, dead-letter queues, and controlled event replay procedures.
- Built asynchronous ingestion pipelines with **Kafka**, **Celery**, AWS SQS, and bulk PostgreSQL operations, absorbing scanning bursts without exhausting database connections or degrading customer-facing analytics workflows.
- Created a reusable React design system with **Storybook**, accessible components, design tokens, typed properties, and visual regression checks, standardizing complex tables, filters, forms, dialogs, and status indicators.
- Established automated quality gates using **Pytest**, **Jest**, **React Testing Library**, **Playwright**, **Pact**, and **Testcontainers**, detecting schema mismatches, migration defects, and interface regressions before production deployment.
- Mentored engineers through architecture reviews, incident retrospectives, pair programming, and performance-focused code reviews while coordinating technical decisions across security, product, frontend, backend, and quality teams.

Cubic Telecom USA — Software Engineer

Remote/ Full time

January 2019 – April 2021

San Diego, CA

- Developed connected-mobility applications using **Python 3.6–3.8**, **Django REST Framework**, **React 15–16**, **TypeScript**, PostgreSQL, and RabbitMQ for device registration, SIM provisioning, subscriptions, billing, and network-status monitoring.
- Improved provisioning throughput by **38%** through asynchronous task execution, bounded concurrency, batched database writes, and provider-specific rate controls that protected external carrier systems during traffic spikes.
- Resolved failed activations caused by inconsistent carrier identifiers and incompatible response structures by creating canonical models, typed adapters, correlation identifiers, validation rules, and replayable reconciliation workflows.
- Reworked slow fleet-management screens with virtualized tables, server-side filtering, normalized state, incremental loading, and obsolete-request cancellation, maintaining responsive interfaces for accounts with large device inventories.
- Designed versioned REST APIs with **Django REST Framework**, OpenAPI documentation, standardized error envelopes, permission-aware endpoints, and stable response contracts consumed by customer portals and operations applications.
- Implemented message-driven provisioning through **Celery**, RabbitMQ, durable queues, dead-letter handling, bounded retries, and idempotent workers, preventing duplicate SIM activations while preserving complete transaction traceability.
- Corrected stale device statuses caused by out-of-order carrier callbacks through timestamp validation, monotonic state transitions, reconciliation jobs, and real-time React updates delivered through WebSocket connections.
- Containerized Python and React applications with **Docker** and standardized CI/CD through **Azure DevOps**, adding automated tests, security checks, migration validation, environment controls, and production approval gates.
- Collaborated with telecom specialists, designers, support teams, and product managers to convert roaming and provisioning rules into maintainable service boundaries, practical interfaces, documented edge cases, and incremental releases.

Asurion — Software Developer*Remote/ Full time***January 2017 – December 2018***Nashville, TN*

- Built customer and technician applications using **Python 3.5–3.7**, **Django REST Framework**, **React 14–16**, Redux, Redis, and SQL Server for policy eligibility, claims, payments, repair scheduling, and status tracking.
- Redesigned a fragmented claim journey as a guided React workflow backed by recoverable Python orchestration APIs, increasing successful online claim completion by **27%** through clearer validation and failure recovery.
- Created a backend-for-frontend layer that combined customer, policy, inventory, payment, and appointment information, shielding React applications from incompatible downstream contracts and partial service outages.
- Corrected duplicate repair bookings caused by repeated submissions and delayed responses through idempotency tokens, database uniqueness constraints, transactional validation, and visible request-state handling.
- Migrated selected jQuery and server-rendered pages to reusable React components while introducing shared form controls, centralized state, route-level organization, and consistent client-side validation behavior.
- Implemented **Redis** caching for frequently requested eligibility and repair-location data, applying explicit invalidation and short expiration periods to improve responsiveness without returning outdated policy decisions.
- Developed **Celery** reconciliation jobs for incomplete claims and delayed payment callbacks, recovering transactions after provider timeouts while preserving audit records for customer-service and finance investigations.
- Expanded automated coverage with **Pytest**, unittest, Jest, Enzyme, Selenium, service mocks, and API integration tests for claims, payments, appointments, eligibility decisions, and recoverable production failures.

Rustici Software — Junior Software Developer*Hybrid/ Full time***June 2015 – December 2016***Nashville, TN*

- Maintained learning-platform services using **Python 2.7 and 3.4**, **Django**, Flask, MySQL, JavaScript, Backbone.js, and jQuery for course imports, SCORM launches, learner-state persistence, and xAPI processing.
- Implemented REST endpoints and administration workflows for course ingestion, launch configuration, and completion settings, adding validation for malformed archives, incomplete manifests, and nonstandard learning-management-system payloads.
- Resolved excessive memory usage during large course imports by streaming compressed files, limiting parallel extraction, releasing temporary resources, and exposing accurate progress and actionable failure messages.
- Built defensive processing for duplicated and out-of-order learning events, preserving accurate completion, score, and resume state across platforms with inconsistent callback and retry behavior.
- Improved database reliability by adding targeted indexes, transactional updates, and repeatable migration scripts for learner-state tables used during high-volume course launches and reporting periods.
- Introduced reusable browser views and evaluated component-based interface patterns for selected administration modules, creating a practical foundation for later adoption of React-based frontend development.
- Added automated tests around course imports, launch tokens, xAPI statements, and completion transitions while collaborating with support specialists to reproduce unusual SCORM integration failures.

EDUCATION

Tennessee State University — Bachelor's Degree in Computer Science

Sep 2011 – May 2015

CERTIFICATION

- **AWS Certified Developer – Associate** — Amazon Web Services
- **Microsoft Certified: Azure Developer Associate** — Microsoft
- **Certified Kubernetes Application Developer — CKAD** — The Linux Foundation and Cloud Native Computing Foundation